Project PIV 2026 - UPDATE

Submission and test server

Each group has an account with a jupyter lab at server http://pivhub.sipg.tecnico.ulisboa.pt

Login with username groupXX (01-44) and the password that you received by email.

Change the password immediatly - launch a terminal and issue passwd command.

In this server you'll see a directory named PART1, PART2. These directories will hold your code as specified below.

You also have one notebook that shows how to use the new VisionIST toolbox. The toolbox is ready and will help generating any type of data you may need to build complex applications. The repository is on github: https://github.com/pivist/VisionIST-PIV/

You do not need to use VisionIST to develop the project. It's just a tool and your code for part1 and part2 will be plain python.

We recommend that you use your own computer (the server is slow with 40 people working on it), but keep a copy there and check if it runs.

We are in the process of installing a powerful GPU so that you can use complex models. News soon

Project Review week 26-30 October

You must schedule your project review. Check your email or fenix page where the link to register. Each interview will be 20-30 min and the group will report the progress. The key topics are:

- Camera Model - your camera calibration !
- Part 1 - By now, part 1 should be almost complete. We covered everything in class, specially in PB Classes we basically desgined the solution step by step
- Anything else you did or thought about.

We will provide feedback on the spot. We won't grade you but you'll get a qualitative "grade": Ok (you're in good shape), Catch Up (your knowledge is shaky you need more effort), Oblivious (you have no idea of what you're doing!)

Of course we will detail each appreciation and recommend study

Check the whole document, there are news down there.

Project Goals

The key learning topics of PIV are:

- Camera Model and estimate cameras from data
- Select and track image keypoints
- Relate images through 2D transformations : Homographies
- Learn 3D reconstruction and 3D point cloud processing
- Deal with outliers
- Estimate both 3D and 2D processing from noisy data and outliers

A project in 3 steps

The project develops in two well defined and mandatory parts plus one application, each building on the previous one.

Part	Input	Output	Core tools
I — Planar tracking	RGB sequence + reference (first frame and court model)	One 3x3 homography per frame	Features, matching, RANSAC, DLT / least squares
II — Depth and 3D	RGB-D sequence + intrinsics	One rigid transformation (R, T) per frame, plus one fused point cloud of the whole scene	Back-projection, Procrustes, RANSAC, ICP
III — Application	(To be Defined) Anything from I and II, plus ready-made systems	A working application	Free choice

Part I — The input is an image sequence and a reference. The sequence may contain frames where the court is partly or entirely absent, and the camera may drift far from the reference, but your code must return a transformation from every frame to the reference.

Part II — We add depth to the RGB image. Each frame comes with a registered depth map and the camera intrinsics, so the scene need no longer be planar: you back-project every frame into a 3D point cloud, estimate a full rigid motion instead of a homography, and fuse all the clouds into a single reconstruction of the scene. This is where 3D data is used to its full extent.

Part III — Time allowing, combine the 2D and 3D data with any extra information (including ready-made detectors and trackers) to build something interesting, useful or sophisticated. For example, following the work with homographies, in the sports setting the obvious candidates are a live minimap of player positions in court coordinates, an automatic in/out call from the ball's landing point, a stabilised "virtual broadcast" view synthesised from a moving camera, or a heat map of court occupation over a whole match. We're working on generating 2D and 3D data for this end.

Part I — Homography estimation from RGB images (final)

Objective: develop a robust homography estimation and tracking pipeline using only RGB data. The input is a video sequence of a match. For every frame *i* your code must deliver the 3x3 matrix that maps frame *i* onto the reference — even when the court is partly or entirely out of view.

Two references are of interest, and you compute both:

1. **The first image of the sequence** — the natural choice for tracking, and the direct analogue of last year's template. Mapping to it lets you stabilise the video.
2. **A scale drawing of the court** — a synthetic top view built from the official dimensions (an ITF singles court is 23.77 m long and 8.23 m wide, doubles 10.97 m, service line 6.40 m from the net). This reference carries real-world units, so the transformation is metric and results are comparable across sequences, cameras and matches.

```mermaid
flowchart LR
  A[Frame i] --> B[Detect<br/>features]
  B --> C[Match to<br/>reference]
  C --> D[RANSAC]
  D --> E[DLT + LSE<br/>refinement]
  E --> F[H to reference<br/>+ court model]
  F --> B
```

The loop closes because the estimate for frame *i* is the best initialisation you have for frame *i*+1.

Typical implementation steps

1. **Feature detection and matching** — a scale- and rotation-invariant detector (SIFT, SURF, ORB), descriptors, and correspondences between the current frame and the reference.
2. **Outlier removal with RANSAC** — minimal sets of four correspondences, scored by inlier consensus, with degenerate (near-collinear) samples rejected.
3. **Homography estimation by least squares** — DLT on the inliers after normalisation of the coordinates, optionally refined as a nonlinear optimisation of the reprojection error.
4. **Anchoring to the court model** — one absolute anchor per sequence, obtained from court landmarks; everything after it follows by composition.
5. **Tracking along the sequence** — direct matching to the reference, frame-to-frame composition, or a hybrid with re-anchoring. Understand what each one costs you in drift and in failure modes.
6. **Frames without a court** — close-ups, crowd shots, replays and cuts. Detect them, decide what to output, and say so in the report.

The sport makes the problem honest. The camera drifts and cuts away, players and their shadows cover the lines, the court is often only half inside the frame, the surface is nearly textureless away from the lines, and the painted lines themselves are highly repetitive — four nearly parallel white segments look alike to any local descriptor. A pipeline that survives this is a pipeline that works.

Code structure and libraries

We include below a description of the code structure but for now the important message is that homographies and outlier removal must not use any library other than numpy and scipy. In other words you can use opencv to read images, compute features, match them but you must implement the DLT and Ransac from scratch.

Minimum set of data

You can find datasets in PIV Google Drive but we encourage you to acquire your own data. Additionally in Appendix A we list several datasets in sports in which you can test your code.

We will update the list in the near future.

Part II — Depth, 3D registration and point cloud fusion

Objective: solve the same tracking problem when the scene is no longer a plane. Each frame now arrives as a registered RGB image and depth map, together with the camera intrinsics K. Back-project every frame into a 3D point cloud, register each cloud to the reference, and fuse them all into **a single point cloud of the scene**, built from the points of every image.

Typical implementation steps

1. **Back-projection** — turn each valid pixel into a 3D point with the intrinsics. Missing, zero and out-of-range depth values must be discarded, not back-projected.
2. **3D correspondences** — match 2D features as in Part I, then lift each match to 3D through its depth. A match whose depth is invalid on either side is not a correspondence.
3. **Rigid transformation by Procrustes** — estimate (R, T) between two corresponding 3D point sets in closed form: centre both sets, take the SVD of the cross-covariance, and guard against the reflection that a naive solution can produce.
4. **Robust estimation with RANSAC** — three correspondences are enough for a minimal sample; score by 3D residual and re-estimate on the inliers.
5. **Refinement with ICP** — the RANSAC solution is a good initialisation, not the answer. Refine it on the dense clouds, point-to-point or point-to-plane.
6. **Fusion** — bring every cloud into the reference frame, merge them, and control the result: voxel downsampling or an equivalent duplicate removal, so the fused cloud stays finite and surfaces do not blur into slabs. Carry the RGB colour of each pixel onto its point.

Drift behaves exactly as in Part I, and worse: registering each frame to its predecessor and composing accumulates error along the whole sequence, and in 3D that error is visible as a smeared or doubled surface in the fused cloud. Registering to the reference where possible, re-anchoring, or closing the loop at the end are all legitimate answers — say which one you chose and show what it bought you.

Data for Part II

We are generating the depth images for sequences and will post them as soon as possible. Note that this part of the project relates to techniques not yet lectured in class.

Early definitions for code

Part 1

Your code must include two files, `main1.py` (`main1.m`) and `part1.py` (`part1.m`).

The running command

```
python main1.py path_to_refdir path_images_dir path_feature_dir
path_output_dir
```

• `path_to_refdir` — directory with the reference data: `templateimg.jpg`, the reference image (normally the first frame of the sequence), and `courtmodel.mat`, a dictionary `{"pts": pts, "names": names}` where `pts` is an (N,2) numpy array with the coordinates of the court landmarks in metres on the court plane, and `names` labels them.
• `path_images_dir` — directory with the image sequence, jpg files named `somename_NNNN.jpg` where NNNN is the sequential number of the image.
• `path_feature_dir` — directory where image features are stored in .mat format, `somename_NNNN.mat`.
• `path_output_dir` — directory for the output results.

main1.py

• May import `cv2`.
• Must import `part1`.
• Is tasked with processing the image files and generating the feature files, stored in `path_feature_dir`. The format of each file is defined by the script distributed earlier, pivist/features — the easy way is to include that code in `main1.py`.
• Should call `part1(args)`, defined next.

part1.py

• Cannot import `cv2` or any package other than the allowed ones (flann or a replacement, numpy, scipy). Other packages may be requested, with justification.
• Must define one function, `part1(path1, path2, path3, path4)`, where the arguments are the inputs of the run command above.
• Must save in `path_output_dir` one file `homography_NNNN.mat` per frame, containing the dictionary `{"H": H, "Hc": Hc}`:
    • `H` is a (3,3) numpy matrix, the homography from frame NNNN to the **reference image**.
    • `Hc` is a (3,3) numpy matrix, the homography from frame NNNN to the **court model**, mapping image pixels to court coordinates in metres.
• One matrix of each kind must be written for every frame in the sequence, including frames where the court is not visible.

Part 2

The input is a reference RGB and depth image, a sequence of RGB images and depth maps, and the camera intrinsics. The output is the rigid transformation from each frame to the reference, **and the fused point cloud**.

Your code must include one file named `main2.py` (`main2.m`).

The running command

```
python main2.py path_to_refdir path_images_dir path_output_dir
```

• `path_to_refdir` — directory with the template data: `templatergb.jpg` and `templatedepth.mat`.
• `path_images_dir` — directory with the RGB and depth sequence, `somename_NNNN.jpg` and `somename_NNNN.mat`, where NNNN is the sequential number of the frame.
• `path_output_dir` — directory for the output results.

Depth files are .mat files with a dictionary `{"depth": depth, "K": K}`, where `depth` is a depth image (numpy matrix) and `K` is the (3,3) matrix of intrinsics.

`main2.py`

• May import `cv2` or other packages, but **not** packages that compute 3D transformations or perform outlier removal. In principle only `cv2` and what Part 1 already allowed.
• Must save in `path_output_dir` one file `transform_NNNN.mat` per frame, with the dictionary `{"R": R, "T": T}`: `R` is a (3,3) numpy matrix, the rotation from frame NNNN to the reference, and `T` is a (3,1) numpy vector, the translation.
• Must save one file `pointcloud.mat` with the dictionary `{"xyz": xyz, "rgb": rgb}`, where `xyz` is an (M,3) array of 3D points expressed in the **reference frame** and in metres, and `rgb` is an (M,3) uint8 array with the colour of each point.
• Must also write `pointcloud.ply`, so the result can be opened directly in MeshLab or CloudCompare.
• The fused cloud must contain points coming from **all** the frames, after duplicate removal. State the voxel size or equivalent criterion you used, and keep the result under about 5 million points.

Part 3

We break this part into two subparts:

3.1 - Part II without depth. You receive a sequence of images (say a video) and you must return "something". In short, what did you learn that can use a sequence of images ? Motion, 3D reconstruction ... We will interact after the project review to make this more specific.

3.2 - Application TBD. Using the VisionIST package we will set a challenge, provided the reviews are decent.

Using VisionIST

The feature extraction step of `main1.py` can be done with the **features box** of VisionIST-PIV, instead of calling OpenCV yourself. The box wraps exactly the SIFT extractor we distributed as pivist/features, so the `.mat` files it produces are the ones `part1.py` expects — same `kp` matrix, same layout.

The features box is **pure CPU**: no GPU, no weights. You can run it on your own laptop (`cd fleet && docker compose up -d features`, host port 9071) or point the client at the PIV server once it is available. The only change is the address.

Install the client: `pip install visionist-client`

What the box returns. Call it with a list of images and it answers with two arrays:

Field	Shape	Meaning
`keypoints`	`(num_images, 2 + 128, N)`	
`counts`	`(num_images,)`	the true number of keypoints of each image — the rest of `N` is padding

Always slice with `counts`: `kp[i, :2, :counts[i]].T` gives the (N_i, 2) points and `kp[i, 2:, :counts[i]].T` the (N_i, 128) descriptors. Writing `kp[i]` whole silently feeds your matcher hundreds of zero descriptors, which match each other perfectly and ruin the RANSAC.

Example `main1.py`

python

```
#!/usr/bin/env python3
"""main1.py — extract features with the VisionIST features box, then
run part1.

Usage:
    python main1.py path_to_refdir path_images_dir path_feature_dir
path_output_dir
"""
import sys
import pathlib
from scipy.io import savemat
from visionist_client import Visionist

from part1 import part1          # your code — no cv2, no opencv
homographies

# Address of the features box: your own machine, or the PIV server.
BOX = "localhost:9071"
BATCH = 16                        # images per call; keep the
payload reasonable

SIFT_PARAMS = {
    "nfeatures": 0,               # 0 = keep all
    "n_octave_layers": 3,
    "contrast_threshold": 0.04,
    "edge_threshold": 10,
    "sigma": 1.6,
    "draw": False,                # set True to also get the
annotated JPEGs
    "mat": False,                 # we write the .mat files
ourselves, below
}


def extract_features(images_dir, feature_dir):
    """Run every image through the features box and write one .mat
per image."""
    images = sorted(pathlib.Path(images_dir).glob("*.jpg"))
    if not images:
        raise SystemExit(f"no jpg files in {images_dir}")

    feature_dir = pathlib.Path(feature_dir)
    feature_dir.mkdir(parents=True, exist_ok=True)

    box = Visionist(BOX)

    for start in range(0, len(images), BATCH):
        chunk = images[start:start + BATCH]
```

```
        res = box.run(
            data={"images": chunk, "names": [p.stem for p in chunk]},
            config={"features": {"command": "extract",
                                 "parameters": SIFT_PARAMS}},
        )

        kp = res.fields["keypoints"]      # (n, 130, N), zero-padded
        counts = res.fields["counts"]     # (n,) true keypoint count

        for i, img in enumerate(chunk):
            n = int(counts[i])
            # Drop the padding. Keep the (2 + 128, n) layout of
pivist/features,
            # so part1 reads exactly the same format as the
distributed script.
            savemat(feature_dir / f"{img.stem}.mat", {"kp": kp[i, :, :n]})
            print(f"{img.name}: {n} keypoints")


def main():
    if len(sys.argv) != 5:
        raise SystemExit(__doc__)

    refdir, images_dir, feature_dir, output_dir = sys.argv[1:5]

    extract_features(images_dir, feature_dir)

    pathlib.Path(output_dir).mkdir(parents=True, exist_ok=True)
    part1(refdir, images_dir, feature_dir, output_dir)


if __name__ == "__main__":
    main()
```

Two things worth noticing. First, `main1.py` never computes a homography — it only produces features and hands the four paths to `part1`, where your own DLT and RANSAC live. Second, nothing above is specific to the box: if you would rather call `cv2.SIFT_create()` directly, replace `extract_features` and leave the rest alone. The contract between `main1.py` and `part1.py` is the directory of `.mat` files, not the tool that filled it.

Appendix A — Evaluation without ground truth

There is no ground-truth transformation for these sequences, and that is deliberate: in a real deployment there never is. Both parts are evaluated on **geometric self-consistency**, which you can compute yourself, on every frame, without anyone labelling anything. Consider the case of images from Tennis (adapt for other datasets).

Part I — homography

Check	What it catches	How to compute	What good looks like
Line reprojection error	A homography that is subtly wrong everywhere	Warp the court model lines into frame *i*; measure the distance from each warped line to the nearest detected white-line pixels	Median under ~2-3 px at 720p on frames where the court is clearly visible
Cycle consistency	Estimates that are individually plausible but mutually inconsistent	For a triplet (i, j, k): transfer a grid of points through H(i,j) H(j,k) H(k,i)	Sub-pixel for nearby frames; growth with baseline is expected and should be reported
Forward-backward consistency	Drift in a composed track		A few pixels over a full rally
Temporal smoothness	Isolated catastrophic frames	Project the four court corners in every frame and plot their velocity; a real camera motion is smooth	Spikes should coincide with cuts you already flagged, and nowhere else
Metric residual	A transformation that is projectively valid but not the court	Map the court to the model and measure known distances — service box width, net length, baseline separation	Within a few centimetres of the official values
		On a handful of frames, mark the	

Warped-court IoU	Gross misregistration	frames; mark the court polygon by hand and compare with the warped model polygon	Above 0.95 on clean frames

Part II — registration and fusion

The same principle, with one advantage: in 3D the fused cloud grades itself, and the errors are in metres.

Check	What it catches	How to compute	What good looks like
Inlier residual	A wrong (R, T)	RMS distance between corresponding 3D inliers after applying the estimated transformation	Millimetres to about a centimetre, depending on sensor and range
Cloud-to-cloud distance	A fit that satisfies the features but not the surface	After registering a frame, median	
Surface thickness	Accumulated drift	Fit a plane to a genuinely flat region of the **fused** cloud and measure the spread of the residuals	A few millimetres. Centimetres means drift
Cycle consistency	Chains of transforms that disagree	Compose around a loop i to j to k to i; report the	Sub-degree and
Rotation validity	A broken Procrustes	Check that R is orthonormal and det(R) = +1 for every output matrix	Exactly. A determinant of -1
Coverage	A fusion that silently drops frames	Fraction of frames	Every frame contributes, unless you flagged it as failed

		downsampling	

Beyond the numbers, produce the pictures. A short video with the court model overlaid on the original frames, a stabilised sequence warped to the first frame, and a top-view minimap showing where the players stood, are worth more than a page of prose — and they make failures impossible to hide. For Part II, show the fused cloud rendered from two or three viewpoints: colouring each point by the frame it came from is the fastest way to expose a misalignment.

Appendix B — Sports Datasets

There are a huge number of datasets in sports where homographies are all you need to locate players in the field. Consider the repositories below as potentially interesting. We will be updating this list.

Tennis

Dataset	Content	Annotations	Camera	Access	Role
TrackNet (paper)	Set 1: 81 clips, one complete rally each, 20,844 frames, 1280x720 at 30 fps. Set 2: 9 matches, 23,903 frames	Ball position and visibility only (CSV)	Broadcast pan-tilt-zoom	Google Drive, no registration, licence unstated	**The video source.** Contiguous, frame-numbered rally clips with the camera motion the project needs
TennisCourtDetector (mirror)	8,841 frames, 1280x720, hard, clay and grass	**14 court keypoints per image** (JSON), ~3-5 px annotation error, plus a top-view court model in `court_reference.py`	Broadcast, but frames sampled	HuggingFace mirror, 7.26 GB, MIT, no access request	**The court**
TennisExpert / TennisVL	202 matches,	Commentary, scoreboard, shot	Broadcast PTZ	GitHub + Google Drive, licence unstated	A quarry for more clips if 81 rallies are not enough
	100-1,400	Court keypoints.			Licence-

Roboflow keypoint sets (1, 2)	isolated frames each	schemas differ between projects	Broadcast frames, no sequences	CC BY 4.0 / MIT	clean supplement, safe to redistribute
TenniSet	5 matches from YouTube	Temporal event labels, commentary alignment	Broadcast	GitHub, MIT code	Not usable — nothing geometric

Two references for the evaluation side, both soccer: TS-WorldCup (3,812 time-sequence frames with per-frame ground-truth homography, 588 MB) and SoccerNet Game State Reconstruction (200 clips of 30 s at 1080p with per-frame pitch localisation). Their metrics — reprojection error in metres, IoU of the warped court polygon — are the ones adopted in Appendix A.

If another sport is preferred

Tennis was chosen for its rich line structure, few players occluding the court, clean rally segmentation and available annotations. Basketball and volleyball remain viable, with different trade-offs.

Sport	Dataset	Content	Annotations	Camera	Verdict
Basketball	DeepSportradar calibration (data)	728 image-calibration	**True camera calibration**: K, R, T, distortion — the strongest geometric ground	Static multi-	Best ground truth, but
Basketball / Volleyball	SportsMOT	240 clips, 25 fps, 1280x720, ~485 frames each, no cuts within a clip	Player bounding boxes only	Fast, variable-speed broadcast motion	Ideal video substrate, court annotation entirely on you. CC BY-NC 4.0
Basketball	TrackID3x3	3x3 full-court video: indoor fixed, outdoor fixed, **outdoor drone**	Player boxes and some pose	The drone subset genuinely translates — unusual and hard	Interesting "hard mode"; half-court geometry is simpler
Volleyball	VB14 (TU Graz)	6 full matches, 1920x1080 at 25 fps, 18,960 annotated frames	Player boxes and 7 activity classes	Single elevated near-static camera	Best volleyball video quality; the near-static camera makes tracking too easy
		55			Short

Volleyball	Ibrahim et al.	broadcast videos, 4,830 target frames,	Player boxes, action and team-activity classes	Broadcast, frequent shot changes	

Basketball gives real calibration ground truth but a static camera; volleyball gives an easy court to annotate but no annotations at all and little camera motion. The schematic top-view reference is free in both cases — the FIBA court is 28x15 m and the FIVB court 18x9 m. One caution for basketball: the three-point line is a circular arc, so it is not a landmark you can intersect like a straight line, and occlusion around the key removes exactly the features you need.